

Perceus: Garbage Free Reference Counting with Reuse

Alex Reinking*
Microsoft Research
Redmond, WA, USA
alex_reinking@berkeley.edu

Leonardo de Moura
Microsoft Research
Redmond, WA, USA
leonardo@microsoft.com

Ningning Xie*
University of Hong Kong
Hong Kong, China
nnxie@cs.hku.hk

Daan Leijen
Microsoft Research
Redmond, WA, USA
daan@microsoft.com

Abstract

We introduce Perceus, an algorithm for precise reference counting with reuse and specialization. Starting from a functional core language with explicit control-flow, Perceus emits precise reference counting instructions such that (cycle-free) programs are *garbage free*, where only live references are retained. This enables further optimizations, like reuse analysis that allows for guaranteed in-place updates at runtime. This in turn enables a novel programming paradigm that we call *functional but in-place* (FBIP). Much like tail-call optimization enables writing loops with regular function calls, reuse analysis enables writing in-place mutating algorithms in a purely functional way. We give a novel formalization of reference counting in a linear resource calculus, and prove that Perceus is sound and garbage free. We show evidence that Perceus, as implemented in Koka, has good performance and is competitive with other state-of-the-art memory collectors.

CCS Concepts: • Software and its engineering → Runtime environments; Garbage collection; • Theory of computation → Linear logic.

Keywords: Reference Counting, Algebraic Effects, Handlers

ACM Reference Format:

Alex Reinking, Ningning Xie, Leonardo de Moura, and Daan Leijen. 2021. Perceus: Garbage Free Reference Counting with Reuse. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '21)*, June 20–25, 2021, Virtual, Canada. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3453483.3454032>

*The first two authors contributed equally to this work.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

PLDI '21, June 20–25, 2021, Virtual, Canada

© 2021 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-8391-2/21/06.

<https://doi.org/10.1145/3453483.3454032>

1 Introduction

Reference counting [7], with its low memory overhead and ease of implementation, used to be a popular technique for automatic memory management. However, the field has broadly moved in favor of generational tracing collectors [31], partly due to various limitations of reference counting, including cycle collection, multi-threaded operations, and expensive in-place updates.

In this work we take a fresh look at reference counting. We consider a programming language design that gives strong compile-time guarantees in order to enable efficient reference counting at run-time. In particular, we build on the pioneering reference counting work in the Lean theorem prover [46], but we view it through the lens of language design, rather than purely as an implementation technique.

We demonstrate our approach in the Koka language [23, 25]: a functional language with mostly immutable data types together with a strong type and effect system. In contrast to the dependently typed Lean language, Koka is general-purpose, with support for exceptions, side effects, and mutable references via general algebraic effects and handlers [39, 40]. Using recent work on evidence translation [50–52], all these control effects are compiled into an internal core language with explicit control flow. Starting from this functional core, we can statically transform the code to enable efficient reference counting at runtime. In particular:

- Due to explicit control flow, the compiler can emit *precise* reference counting instructions where a (non-cyclic) reference is dropped as soon as possible. We call this *garbage free* reference counting as only live data is retained (§ 2.2).
- We show that precise reference counting enables many optimizations, in particular *drop specialization* which removes many reference count operations in the fast path (Section 2.3), *reuse analysis* which updates (immutable) data in-place when possible (Section 2.4), and *reuse specialization* which removes many in-place field updates (Section 2.5). The reuse analysis shows the benefit of a holistic approach: even though the surface language has immutable data types with strong guarantees, we can use dynamic run-time information, e.g. whether a reference is unique, to update in-place when possible.

- The in-place update optimization is guaranteed, which leads to a new programming paradigm that we call *FBIP: functional but in-place* (Section 2.6). Just like tail-call optimization lets us write loops with regular function calls, reuse analysis lets us write in-place mutating algorithms in a purely functional way. We showcase this approach by implementing a functional version of in-order Morris tree traversal [35], which is stack-less, using in-place tree node mutation via FBIP.
- We present a formalization of general reference counting using a novel linear resource calculus, λ^1 , which is closely based on linear logic (Section 3), and we prove that reference counting is sound for any program in the linear resource calculus. We then present the *Perceus*¹ algorithm as a deterministic syntax-directed version of λ^1 , and prove that it is both *sound* (i.e. never drops a live reference), and *garbage free* (i.e. only retains reachable references).
- We demonstrate Perceus by providing a full implementation for the strongly typed functional language Koka [1]. The implementation supports typed algebraic effect handlers using evidence translation [51] and compiles into standard C11 code. The use of reference counting means no runtime system is needed and Koka programs can readily link with other C/C++ libraries.
- We show evidence that Perceus, as implemented for Koka, competes with other state-of-the-art memory collectors (Section 4). We compare our implementation in allocation intensive benchmarks against OCaml, Haskell, Swift, and Java, and for some benchmarks to C++ as well. Even though the current Koka compiler does not have many optimizations (besides the ones for reference counting), it has outstanding performance compared to these mature systems. As a highlight, on the tree insertion benchmark, the purely functional Koka implementation is within 10% of the performance of the in-place mutating algorithm in C++ (using `std::map` [13]).

Even though we focus on Koka in this paper, we believe that Perceus, and the FBIP programming paradigm we identify, are both broadly applicable to other programming languages with similar static guarantees for explicit control flow.

There is an accompanying technical report [41] containing all the proofs and further benchmark results.

2 Overview

Compared to a generational tracing collector, reference counting has low memory overhead and is straightforward to implement. However, while the cost of tracing collectors is linear in the live data, the cost of reference counting is linear in the number of reference counting operations. Optimizing the total cost of reference counting operations is therefore our main priority. There are at least three known

problems that make reference counting operations expensive in practice and generally inferior to tracing collectors:

- *Concurrency*: when multiple threads share a data structure, reference count operations need to be atomic, which is expensive.
- *Precision*: common reference counted systems are not *precise* and hold on to objects too long. This increases memory usage and prevents aggressive optimization of many reference count operations.
- *Cycles*: if object references form a cycle, the runtime needs to handle them separately, which re-introduces many of the drawbacks of a tracing collector.

We handle each of these issues in the context of an eager, functional language using immutable data types together with a strong type and effect system. For concurrency, we precisely track when objects can become thread-shared (Section 2.7.2). For precision, we introduce Perceus, our algorithm for inserting precise reference counting operations that can be aggressively optimized. In particular, we eliminate and fuse many reference count operations with *drop specialization* (Section 2.3), turn functional matching into in-place updates with *reuse analysis* (Section 2.4), and minimize field updates with *reuse specialization* (Section 2.5).

Finally, although we currently do not supply a cycle collector, our design has two mitigations that reduces the occurrences of cycles in the first place. First, *(co)inductive* data types and eager evaluation prevent cycles outside of explicit mutable references, and it is statically known where cycles can possibly be introduced in the code (Section 2.7.4). Second, being a mostly functional language, mutable references are not often used – moreover, reuse analysis greatly reduces the need for them since in-place mutation is typically inferred.

The reference count optimizations are our main contribution and we start with a detailed overview in the following sections, ending with details about how we mitigate the impact of concurrency and cycles.

2.1 Types and Effects

We start with a brief introduction to Koka [23, 25] – a strongly typed, functional language that tracks all (side) effects. For example, we can define a squaring function as:

```
fun square( x : int ) : total int { x * x }
```

Here we see two types in the result: the effect type `total` and the result type `int`. The `total` type signifies that the function can be modeled semantically as a mathematically *total* function, which always terminates without raising an exception (or having any other observable side effect). Effectful functions get more interesting effect types, like:

```
fun println( s : string ) : console ()
fun divide( x : int, y : int ) : exn int
```

where `println` has a `console` effect and `divide` may raise an exception (`exn`) when dividing by zero. It is beyond the scope of this paper to go into full detail, but a novel feature of Koka

¹Perceus, pronounced *per-see-us*, is a loose acronym of “PrEcise Reference Counting with rEUse and Specialization”.

is that it supports *typed algebraic effect handlers* which can define new effects like `async/await`, iterators, or co-routines without needing to extend the language itself [24–26].

Koka uses algebraic data types extensively. For example, we can define a polymorphic list of elements of type `a` as:

```
type list(a) {
  Cons( head : a, tail : list(a) )
  Nil
}
```

We can match on a list to define a polymorphic `map` function that applies a function `f` to each element of a list `xs`:

```
fun map( xs : list(a), f : a -> e b ) : e list(b) {
  match(xs) {
    Cons(x, xx) -> Cons(f(x), map(xx, f))
    Nil        -> Nil
  }
}
```

Here we transform the list of generic elements of type `a` to a list of generic elements of type `b`. Since `map` itself has no intrinsic effect, the overall effect of `map` is polymorphic, and equals the effect `e` of the function `f` as it is applied to every element. The `map` function demonstrates many interesting aspects of reference counting and we use it as a running example in the following sections.

2.2 Precise Reference Counting

An important attribute that sets Perceus apart is that it is *precise*: an object is freed as soon as no more references remain. By contrast, common reference counting implementations tie the liveness of a reference to its lexical scope, which might retain memory longer than needed. Consider:

```
fun foo() {
  val xs = list(1, 1000000) // create large list
  val ys = map(xs, inc)     // increment elements
  print(ys)
}
```

Many compilers emit code similar to:

```
fun foo() {
  val xs = list(1, 1000000)
  val ys = map(xs, inc)
  print(ys)
  drop(xs)
  drop(ys)
}
```

where we use a gray background for generated operations. The `drop(xs)` operation decrements the reference count of an object and, if it drops to zero, recursively drops all children of the object and frees its memory. These “scoped lifetime” reference counts are used by the C++ `shared_ptr<T>` (calling the destructor at the end of the scope), Rust’s `Rc<T>` (using the `Drop` trait), and Nim (using a `finally` block to call `destroy`) [53]. It is not required by the semantics, but Swift typically emits code like this as well [14].

Implementing reference counting this way is straightforward and integrates well with exception handling where the drop operations are performed as part of stack unwinding. But from a performance perspective, the technique is not always optimal: in the previous example, the large list `xs` is

retained in memory while a new list `ys` is built. Both exist for the duration of `print`, after which a long, cascading chain of drop operations happens for each element in each list.

Perceus takes a more aggressive approach where *ownership* of references is passed down into each function: now `map` is in charge of freeing `xs`, and `ys` is freed by `print`: no `drop` operations are emitted inside `foo` as all local variables are *consumed* by other functions, while the `map` and `print` functions drop the list elements as they go. In this example, Perceus generates the code for `map` as given in Figure 1b. In the `Cons` branch, first the head and tail of the list are *dupped*, where a `dup(x)` operation increments the reference count of an object and returns itself. The `drop(xs)` then frees the initial list node. We need to `dup f` as well as it is used twice, while `x` and `xx` are consumed by `f` and `map` respectively.

At first blush, this seems more expensive than the scoped approach but, as we will see, this change enables many further optimizations. More importantly, transferring ownership, rather than retaining it, means we can free an object immediately when no more references remain. This both increases cache locality and decreases memory usage. For `map`, the memory usage is halved: the list `xs` is deallocated while the new list `ys` is being allocated.

2.3 Drop Specialization

Once we change to precise, ownership-based reference counting, there are many further optimization opportunities. After the initial insertion of `dup` and `drop` operations, we perform a *drop specialization* pass. The basic `drop` operation is defined in pseudocode as:

```
fun drop( x ) {
  if (is-unique(x)) then drop children of x; free(x)
  else decref(x)
}
```

and drop specialization essentially inlines the `drop` operation specialized at a specific constructor. Figure 1c shows the drop specialization of our `map` example. Note that we only apply drop specialization if the children are used, so no specialization takes place in the `Nil` branch.

Again, it appears we made things worse with extra operations in each branch, but we can perform another transformation where we push down `dup` operations into branches followed by standard *dup/drop fusion* where corresponding `dup/drop` pairs are removed. Figure 1d shows the code that is generated for our `map` example.

After this transformation, almost all reference count operations in the fast path are gone. In our example, every node in the list `xs` that we map over is unique (with a reference count of 1) and so the `if (is-unique(xs))` test always succeeds, thus immediately freeing the node without any further reference counting.

2.4 Reuse Analysis

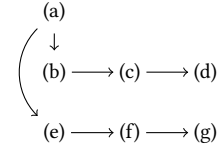
There is more we can do. Instead of freeing `xs` and immediately allocating a fresh `Cons` node, we can try to *reuse* `xs`

```

fun map( xs : list(a), f : a -> e b ) : e list(b) {
  match(xs) {
    Cons(x,xx) -> Cons(f(x), map(xx,f))
    Nil        -> Nil
  }
}

```

(a) A polymorphic map function



```

fun map( xs, f ) {
  match(xs) {
    Cons(x,xx) {
      dup(x); dup(xx); drop(xs)
      Cons( dup(f)(x), map(xx, f))
    }
    Nil { drop(xs); drop(f); Nil }
  }
}

```

(b) dup/drop insertion (2.2)

```

fun map( xs, f ) {
  match(xs) {
    Cons(x,xx) {
      dup(x); dup(xx)
      if (is-unique(xs))
        then drop(x); drop(xx); free(xs)
        else decref(xs)
      Cons( dup(f)(x), map(xx, f))
    }
    Nil { drop(xs); drop(f); Nil }
  }
}

```

(c) drop specialization (2.3)

```

fun map( xs, f ) {
  match(xs) {
    Cons(x,xx) {
      if (is-unique(xs))
        then free(xs)
        else dup(x); dup(xx); decref(xs)
      Cons( dup(f)(x), map(xx, f))
    }
    Nil { drop(xs); drop(f); Nil }
  }
}

```

(d) push down dup and fusion (2.3)

```

fun map( xs, f ) {
  match(xs) {
    Cons(x,xx) {
      dup(x); dup(xx);
      val ru = drop-reuse(xs)
      Cons@ru( dup(f)(x), map(xx, f))
    }
    Nil { drop(xs); drop(f); Nil }
  }
}

```

(e) reuse token insertion (2.4)

```

fun map( xs, f ) {
  match(xs) {
    Cons(x,xx) {
      dup(x); dup(xx);
      val ru = if (is-unique(xs))
        then drop(x); drop(xx); &xs
        else decref(xs); NULL
      Cons@ru( dup(f)(x), map(xx, f))
    }
    Nil { drop(xs); drop(f); Nil }
  }
}

```

(f) drop-reuse specialization (2.4)

```

fun map( xs, f ) {
  match(xs) {
    Cons(x,xx) {
      val ru = if (is-unique(xs))
        then &xs
        else dup(x); dup(xx);
        decref(xs); NULL
      Cons@ru( dup(f)(x), map(xx, f))
    }
    Nil { drop(xs); drop(f); Nil }
  }
}

```

(g) push down dup and fusion (2.4)

Fig. 1. Drop specialization and reuse analysis for map.

directly as first described by Ullrich and de Moura [46]. *Reuse analysis* is performed before emitting the initial reference counting operations. It analyses each `match` branch, and tries to pair each matched pattern to allocated constructors of the same size in the branch. In our `map` example, `xs` is paired with the `Cons` constructor. When such pairs are found, and the matched object is not live, we generate a `drop-reuse` operation that returns a *reuse token* that we attach to any constructor paired with it:

```

fun map( xs, f ) {
  match(xs) {
    Cons(x,xx) {
      val ru = drop-reuse(xs)
      Cons@ru( f(x), map(xx, f))
    }
    Nil -> Nil
  }
}

```

The `Cons@ru` annotation means that (at runtime) if `ru=NULL` then the `Cons` node is allocated fresh, and otherwise the memory at `ru` is of the right size and can be used directly. Figure 1e shows the generated code after reference count insertion. Compared to the program in Figure 1b, the generated code now consumes `xs` using `drop-reuse(xs)` instead of `drop(xs)`.

Just like with *drop specialization* we can also specialize *drop-reuse*. The *drop-reuse* operation is specified in pseudocode as:

```

fun drop-reuse( x ) {
  if (is-unique(x)) then drop children of x; &x
  else decref(x); NULL
}

```

where `&x` returns the address of `x`. Figure 1f shows the code for `map` after specializing the *drop-reuse*. Again, we can push down and fuse the `dup` operations, which finally results in the code shown in Figure 1g. In the fast path, where `xs` is uniquely owned, there are no more reference counting operations at all! Furthermore, the memory of `xs` is directly reused to provide the memory for the `Cons` node for the returned list – effectively updating the list *in-place*.

2.5 Reuse Specialization

The final transformation we apply is *reuse specialization*, by which we can further reuse unchanged fields of a constructor. A constructor expression like `Cons@ru(x, xx)` is implemented in pseudocode as:

```

fun Cons@ru( x, xx ) {
  if (ru!=NULL)
    then { ru->head := x; ru->tail := xx; ru } // in-place
  else Cons(x,xx) // malloc'd
}

```

However, for our `map` example there would be no benefit to specializing as all fields are assigned. Thus, we only specialize constructors if at least one of the fields stays the same. As

an example, we consider insertion into a red-black tree [17]. We define red-black trees as:

```
type color { Red; Black }
type tree {
  Leaf
  Node(color: color, left: tree, key: int,
        value: bool, right: tree)
}
```

The red-black tree has the invariant that the number of black nodes from the root to any of the leaves is the same, and that a red node is never a parent of red node. Together this ensures that the trees are always balanced. When inserting nodes, the invariants need to be maintained by rebalancing the nodes when needed. Okasaki’s algorithm [37] implements this elegantly and functionally (the full algorithm can be found in accompanying technical report [41]):

```
fun bal-left( l : tree, k : int, v : bool, r : tree ) : tree {
  match(l) {
    Node(, Node(Red, lx, kx, vx, rx), ky, vy, ry)
    -> Node(Red, Node(Black, lx, kx, vx, rx), ky, vy,
            Node(Black, ry, k, v, r))
    ...
  }
  fun ins( t : tree, k : int, v : bool ) : tree {
    match(t) {
      Leaf -> Node(Red, Leaf, k, v, Leaf)
      Node(Red, l, kx, vx, r) // second branch
      -> if (k < kx) then Node(Red, ins(l, k, v), kx, vx, r)
      ...
      Node(Black, l, kx, vx, r)
      -> if (k < kx && is-red(l))
          then bal-left(ins(l,k,v), kx, vx, r)
      ...
    }
  }
```

For this kind of program, reuse specialization is effective. For example, if we look at the second branch in `ins` we see that the newly allocated `Node` has almost all of the same fields as `t` except for the left tree `l` which becomes `ins(l,k,v)`. After reuse specialization, this branch becomes:

```
Node(Red, l, kx, vx, r) { // second branch
  val ru = if (is-unique(t)) then &t
            else { dup(l); dup(kx); dup(vx); dup(r); NULL }
  if (dup(k) < dup(kx)) {
    val y = ins(l,k,v)
    if (ru!=NULL) then { ru->left := y; ru } // fast path
    else Node(Red, y, kx, vx, r)
  }
```

In the fast path, where `t` is uniquely owned, `t` is reused directly, and only its left child is re-assigned as all other fields stay unchanged. This applies to many branches in this example and saves many assignments.

Moreover, the compiler inlines the `bal-left` function. At that point, every matched `Node` constructor has a corresponding `Node` allocation – if we consider all branches we can see that we either match one `Node` and allocate one, or we match three nodes deep and allocate three. With *reuse analysis* this means that every `Node` is reused in the fast path without doing any allocations!

Essentially this means that for a unique tree, the purely functional algorithm above adapts at runtime to an in-place

```
void inorder( tree* root, void (*f)(tree* t) ) {
  tree* cursor = root;
  while (cursor != NULL /* Tip */) {
    if (cursor->left == NULL) {
      // no left tree, go down the right
      f(cursor->value);
      cursor = cursor->right;
    } else {
      // has a left tree
      tree* pre = cursor->left; // find the predecessor
      while(pre->right != NULL && pre->right != cursor) {
        pre = pre->right;
      }
      if (pre->right == NULL) {
        // first visit, remember to visit right tree
        pre->right = cursor;
        cursor = cursor->left;
      } else {
        // already set, restore
        f(cursor->value);
        pre->right = NULL;
        cursor = cursor->right;
      }
    }
  }
}
```

Fig. 2. Morris in-order tree traversal algorithm in C.

mutating re-balancing algorithm (without any further allocation). Moreover, if we use the tree *persistently* [36], and the tree is shared or has shared parts, the algorithm adapts to copying exactly the shared *spine* of the tree (and no more), while still rebalancing in place for any unshared parts.

2.6 A New Paradigm: Functional but In-Place (FBIP)

The previous red-black tree rebalancing showed that with Perceus we can write algorithms that dynamically adapt to use in-place mutation when possible (and use copying when used persistently). Importantly, a programmer can rely on this optimization happening, e.g. they can see the `match` patterns and match them to constructors in each branch.

This style of programming leads to a new paradigm that we call FBIP: “functional but in place”. Just like tail-call optimization lets us describe loops in terms of regular function calls, reuse analysis lets us describe in-place mutating imperative algorithms in a purely functional way (and get persistence as well). Consider mapping a function `f` over all elements in a binary tree in-order:

```
type tree {
  Tip
  Bin( left: tree, value : int, right: tree )
}
fun tmap( t : tree, f : int -> int ) : tree {
  match(t) {
    Bin(l,x,r) -> Bin( tmap(l,f), f(x), tmap(r,f) )
    Tip -> Tip
  }
}
```

This is already quite efficient as all the `Bin` and `Tip` nodes are reused in-place when `t` is unique. However, the `tmap` function is not tail-recursive and thus uses as much stack space as the depth of the tree.

In 1968, Knuth posed the problem of visiting a tree in-order while using no extra stack- or heap space [22] (For readers not familiar with the problem it might be fun to try

```

type visitor {
  Done
  BinR( right:tree, value : int, visit : visitor )
  BinL( left:tree, value : int, visit : visitor )
}
type direction { Up; Down }

fun tmap( f : int -> int, t : tree,
  visit : visitor, d : direction ) : tree {
  match(d) {
    Down -> match(t) { // going down a left spine
      Bin(1,x,r) -> tmap(f,1,BinR(r,x,visit),Down) // A
      Tip -> tmap(f,Tip,visit,Up) // B
    }
    Up -> match(visit) { // go up through the visitor
      Done -> t // C
      BinR(r,x,v) -> tmap(f,r,BinL(t,f(x),v),Down) // D
      BinL(l,x,v) -> tmap(f,Bin(l,x,t),v,Up) // E
    }
  }
}

```

Fig. 3. FBIP in-order tree traversal algorithm in Koka.

this in your favorite imperative language first and see that it is not easy to do). Since then, numerous solutions have appeared in the literature. A particularly elegant solution was proposed by Morris [35]. This is an in-place mutating algorithm that swaps pointers in the tree to “remember” which parts are unvisited. It is beyond this paper to give a full explanation, but a C implementation is shown in Figure 2. The traversal essentially uses a *right-threaded* tree to keep track of which nodes to visit. The algorithm is subtle, though. Since it transforms the tree into an intermediate graph, we need to state invariants over the so-called *Morris loops* [29] to prove its correctness.

We can derive a functional and more intuitive solution using the FBIP technique. We start by defining an explicit *visitor* data structure that keeps track of which parts of the tree we still need to visit. In Koka we define this data type as `visitor` given in Figure 3. (Interestingly, our visitor data type can be generically derived as a list of the derivative of the tree data type² [20, 30]). We also keep track of which *direction* we are going, either `Up` or `Down` the tree.

We start our traversal by going downward into the tree with an empty visitor, expressed as `tmap(f, t, Done, Down)`. The key idea is that we are either `Done` (C), or, on going downward in a left spine we remember all the right trees we still need to visit in a `BinR` (A) or, going upward again (B), we remember the left tree that we just constructed as a `BinL` while visiting right trees (D). When we come back (E), we restore the original tree with the result values. Note that we apply the function `f` to the saved value in branch D (as we visit *in-order*), but the functional implementation makes it easy to specify a *pre-order* traversal by applying `f` in branch A, or a *post-order* traversal by applying `f` in branch E.

²Conor McBride [30] describes how we can generically derive a *zipper* [20] visitor for any recursive type $\mu x. F$ as a list of the derivative of that type, namely $\text{list}(\frac{\partial}{\partial x} F \mid x = \mu x.F)$. In our case, calculating the derivative of the inductive `tree`, we get $\mu x. 1 + (\text{tree} \times \text{int} \times x) + (\text{tree} \times \text{int} \times x)$, which corresponds to the `visitor` datatype.

Looking at each branch we can see that each `Bin` matches up with a `BinR`, each `BinR` with a `BinL`, and finally each `BinL` with a `Bin`. Since they all have the same size, if the tree is unique, each branch updates the tree nodes *in-place* at runtime without any allocation, where the `visitor` structure is effectively overlaid over the tree nodes while traversing the tree. Since all `tmap` calls are tail calls, this also compiles to a loop and thus needs no extra stack- or heap space.

Finally, just like with re-balancing tree insertion, the algorithm as specified is still purely functional: it uses in-place updating when a unique tree is passed, but it also adapts gracefully to the persistent case where the input tree is shared, or where parts of the input tree are shared, making a single copy of those parts of the tree.

2.7 Static Guarantees and Language Features

So far we have shown that precise reference counting enables powerful analyses and optimizations of the reference counting operations. In this section, we use Koka as an example to discuss how strong static guarantees at compile-time can further allow the precise reference counting approach to be integrated with non-trivial language features.

2.7.1 Non-Linear Control Flow. An essential requirement of our approach is that programs have explicit control flow so that it is possible to statically determine where to insert `dup` and `drop` operations. However, it is in tension with functions that have non-linear control flow, e.g. may throw an exception, use a `longjmp`, or create an asynchronous continuation that is never resumed. For example, if we look at the code for `map` before applying optimizations, we have:

```

fun map( xs, f ) {
  match(xs) {
    Cons(x,xx) {
      dup(x); dup(xx); drop(xs); dup(f)
      Cons( f(x), map(xx, f) )
    }
    ...
  }
}

```

If `f` raised an exception and directly exited the scope of `map`, then `xx` and `f` would leak and never be dropped. This is one reason why a C++ `shared_ptr` is tied to lexical scope; it integrates nicely with the stack unwinding mechanism for exceptions that guarantees each `shared_ptr` is dropped eventually.

In Koka, we guarantee that all control-flow is compiled to explicit control-flow, so our reference count analysis does not have to take non-linear control-flow into account. This is achieved through *effect typing* (Section 2.1) where every function has an effect type that signifies if it can throw exceptions or not. Functions that can throw are compiled into functions that return with an explicit error type that is either `Ok`, or `Error` if an exception is thrown. This is checked and propagated at every invocation³.

³Koka actually generalizes this using a multi-prompt delimited control monad that works for any control effect, with essentially the same principle.

For example, for `map` the compiled code (before optimization) becomes like:

```
fun map( xs, f ) {
  match(xs) {
    Cons(x,xx) {
      dup(x); dup(xx); drop(xs); dup(f)
      match(f(x)) {
        Error(err) -> { drop(xx); drop(f); Error(err); }
        Ok(y) -> { match(map(xx, f)) {
          Error(err) -> drop(y); Error(err)
          Ok(ys) -> Cons(y,ys)
        }
      }
    }
  }
}
```

At this point all errors are explicitly propagated and all control-flow is explicit again. Note that we have no reference count operations on the `error` values as these are implemented as *value* types which are not heap allocated.

This is similar to error handling in Swift [21] (although it requires the programmer to insert a `try` at every invocation), and also similar to various C++ proposals [44] where exceptions become explicit error values.

The example here is specialized for exceptions but the actual Koka implementation uses a generalized version of this technique to implement a multi-prompt delimited control monad [18] instead, which is used in combination with evidence translation [51] to express general algebraic effect handlers (which in turn subsume all other control effects, like exceptions, `async/await`, probabilistic programming, etc).

2.7.2 Concurrent Execution. If multiple threads share a reference to a value, the reference count needs to be incremented and decremented using atomic operations which can be expensive. Ungar et al. [47] report slowdowns up to 50% when atomic reference counting operations are used. Nevertheless, in languages with unrestricted multi-threading, like Swift, almost all reference count operations need to assume that references are potentially thread-shared.

In Koka, the strong type system gives us additional guarantees about which variables may need atomic reference count operations. Following the solution of Ullrich and de Moura [46], we mark each object with whether it can be thread-shared or not, and supply an internal polymorphic operation `tshare : forall a. a -> io ()` which marks any object and its children recursively as being thread-shared. Even though marking is linear, it happens at most once for any object since shared objects cannot be unshared. All objects start out as unshared, and are only marked through explicit operations. In particular, when starting a new thread, the argument passed to the thread is marked as thread-shared. The only other operation that can cause thread sharing is setting a thread-shared mutable reference but this is quite uncommon in typical Koka code. The `drop` and `dup` operations can be implemented efficiently by avoiding atomic operations in the fast path by checking the thread-shared flag.

For example, `drop` may be implemented in C as:

```
static inline void drop( block_t* b ) {
```

```
  if (b->header.thread_shared) {
    if (atomic_dec(&b->header.rc) == 1) drop_free(b);
  } else if (b->header.rc-- == 1) drop_free(b);
}
```

However, this may still present quite some overhead as many `drop` operations are emitted.

In Koka we encode the reference count for thread-shared objects as a negative value. This enables us to use a *single* inlined test to see if we need to take the slow path for either a thread-shared object or an object that needs to be freed; and we can use a fast inlined path for the common case⁴:

```
static inline void drop( block_t* b ) {
  if (b->header.rc <= 1) drop_check(b); // slow path
  else b->header.rc--;
}
```

The `drop_check` function checks if the reference count is 1 to release it, or otherwise it adjusts the reference count atomically. We also use the negative values to implement a *sticky* range where very large reference counts (2^{30} in our implementation) stay without being further adjusted (preventing overflow, and keeping them alive for the rest of the program).

2.7.3 Mutation. Mutation in Koka is done through explicit mutable references. Here we look at first-class mutable reference cells, but Koka also has second-class mutable local variables that can be more convenient. A mutable reference cell is created with `ref`, dereferenced with `!` and updated using `:=`:

```
fun ref( init : a ) : st(h) ref(h,a)
fun (!)( r : ref(h,a) ) : st(h) a
fun (:=)( r : ref(h,a), x : a ) : st(h) ()
```

where each operation has a stateful effect `st(h)` in some heap `h`. A reference cell of type `ref(h,a)` is a first-class *value* that contains a reference to a value of type `a`. As such, there are always two reference counts involved: that of the reference itself, and that of value that is referenced.

When a mutable reference cell is thread-shared, this presents a problem as an update operation may *race* with a read operation to update the reference counts. The pseudocode implementation of both operations is:

```
fun (!)( r ) {      fun (:=)( r, x ) {
  val x = r->value    val y = r->value
  dup(x)             r->value := x
  x                  drop y
}
```

The read operation `!` first reads the current reference in `x`, and then increments its reference count. Suppose though that before the `dup`, the thread is suspended and another thread writes to the same reference: it will read the same object into `y`, update the reference, and then drop `y` – and if `y` has a reference count of 1 it will be freed! When the other thread resumes, it will now try to `dup` the just-freed object.

⁴Since the thread-shared sign-bit is *stable*, we can do the test `b->header.rc <= 1` without needing expensive atomic operations and can use a `memory_order_relaxed` atomic read.

To make this work correctly, we need to perform both operations atomically, either through a double-CAS [9], using hazard pointers [15, 33], or using some other locking mechanism. Either way, this can be quite expensive. Fortunately, in our setting, we can avoid the slow path in most cases. First of all, since FBIP allows for the efficiency of in-place updates with a purely functional specification (Section 2.6), we expect mutable references to be a last resort rather than the default. Secondly, as discussed in Section 2.7.2, we can also check if a mutable reference is actually thread-shared and thus avoid the atomic code path almost all of the time.

2.7.4 Cycles. A known limitation of reference counting is that it cannot release cyclic data structures. Just like with mutability, we try to mitigate its performance impact by reducing the potential for this to occur in the first place. In Koka, almost all data types are immutable and either *inductive* or *coinductive*. It can be shown that such data types are never cyclic (and functions that recurse over such data types always terminate).

In practice, mutable references are the main way to construct cyclic data. Since mutable references are uncommon in our setting, we leave the responsibility to the programmer to break cycles by explicitly clearing a reference cell that may be part of a cycle. Since this strategy is also used by Swift, a widely used language where most object fields are mutable, we believe this is a reasonable approach to take for now. However, we have plans for future improvements: since we know statically that only mutable references are able to form a cycle, we could generate code that tracks those data types at run time and may perform a more efficient form of incremental cycle collection.

2.7.5 Summary. In summary, we have shown how static guarantees at compile-time can be used to mitigate the performance impact of concurrency and the risk of cycles. This paper does not yet present a general solution to all problems with reference counting and future work is required to explore how cycles can be handled more efficiently, and how well Perceus can be used with implicit control flow. Yet, we expect that our approach gives new insights in the general design space of reference counting, and showcase that precise reference counting can be a viable alternative to other approaches. In practice, we found that Perceus has good performance, which is discussed in Section 4.

3 A Linear Resource Calculus

In this section we present a novel linear resource calculus, λ^1 , which is closely based on linear logic. The operational semantics of λ^1 is formalized in an explicit heap with reference counting, and we prove that the operational semantics is sound. We then formalize Perceus as a sound and precise syntax-directed algorithm of λ^1 and thus provide a theoretic foundation for Perceus.

Expressions

$e ::= v \mid e e$	(value, application)
$\mid \text{val } x = e; e$	(bind)
$\mid \text{match } x \{ \overline{p_i \mapsto e_i} \}$	(match)
$\mid \text{dup } x; e$	(duplicate)
$\mid \text{drop } x; e$	(drop)
$\mid \text{match } e \{ \overline{p_i \mapsto e_i} \}$	(match expr)
$v ::= x \mid \lambda x. e$	(variables, functions)
$\mid C v_1 \dots v_n$	(constructor of arity n)
$p ::= C b_1 \dots b_n$	(pattern)
$b ::= x \mid _$	(binder or wildcard)

Contexts $\Delta, \Gamma ::= \emptyset \mid \Delta \cup x$

Syntactic shorthands

$e_1; e_2 \triangleq \text{val } x = e_1; e_2$	sequence, $x \notin \text{fv}(e_2)$
$\lambda _. e \triangleq \lambda x. e$	$x \notin \text{fv}(e)$
$\lambda x. e \triangleq \lambda^{ys} x. e$	$ys = \text{fv}(e)$

Fig. 4. Syntax of the linear resource calculus λ^1 .

$\Delta \mid \Gamma \vdash e \rightsquigarrow e'$	($\uparrow$ is input, while $\downarrow$ is output)
$\frac{}{\Delta \mid x \vdash x \rightsquigarrow x}$	[VAR]
$\frac{\Delta \mid \Gamma, x \vdash e \rightsquigarrow e' \quad x \in \Delta, \Gamma}{\Delta \mid \Gamma \vdash e \rightsquigarrow \text{dup } x; e'}$	[DUP]
$\frac{\Delta \mid \Gamma \vdash e \rightsquigarrow e'}{\Delta \mid \Gamma, x \vdash e \rightsquigarrow \text{drop } x; e'}$	[DROP]
$\frac{\Delta, \Gamma_2 \mid \Gamma_1 \vdash e_1 \rightsquigarrow e'_1 \quad \Delta \mid \Gamma_2 \vdash e_2 \rightsquigarrow e'_2}{\Delta \mid \Gamma_1, \Gamma_2 \vdash e_1 e_2 \rightsquigarrow e'_1 e'_2}$	[APP]
$\frac{\emptyset \mid \Gamma, x \vdash e \rightsquigarrow e' \quad \Gamma = \text{fv}(\lambda x. e)}{\Delta \mid \Gamma \vdash \lambda x. e \rightsquigarrow \lambda^\Gamma x. e'}$	[LAM]
$\frac{x \notin \Delta, \Gamma_1, \Gamma_2 \quad \Delta, \Gamma_2 \mid \Gamma_1 \vdash e_1 \rightsquigarrow e'_1 \quad \Delta \mid \Gamma_2, x \vdash e_2 \rightsquigarrow e'_2}{\Delta \mid \Gamma_1, \Gamma_2 \vdash \text{val } x = e_1; e_2 \rightsquigarrow \text{val } x = e'_1; e'_2}$	[BIND]
$\frac{[\text{MATCH}] \quad \Delta \mid \Gamma, \text{bv}(p_i) \vdash e_i \rightsquigarrow e'_i}{\Delta \mid \Gamma, x \vdash \text{match } x \{ \overline{p_i \mapsto e_i} \} \rightsquigarrow \text{match } x \{ \overline{p_i \mapsto e'_i} \}}$	
$\frac{\Delta, \Gamma_{i+1}, \dots, \Gamma_n \mid \Gamma_i \vdash v_i \rightsquigarrow v'_i \quad 1 \leq i \leq n}{\Delta \mid \Gamma_1, \dots, \Gamma_n \vdash C v_1 \dots v_n \rightsquigarrow C v'_1 \dots v'_n}$	[CON]

Fig. 5. Declarative linear resource rules of λ^1 .

3.1 Syntax

Figure 4 defines the syntax of our linear resource calculus λ^1 . It is essentially an untyped lambda calculus extended with explicit binding as $\text{val } x = e_1; e_2$, and pattern matching as

$$\begin{array}{c}
E ::= \square \mid E \ e \mid v \ E \\
\mid \text{val } x = E; e \\
\\
\frac{e \longrightarrow e'}{E[e] \mapsto E[e']} \text{ [EVAL]} \\
\\
\begin{array}{ll}
(\text{app}) & (\lambda x. e) \ v \longrightarrow e[x:=v] \\
(\text{bind}) & \text{val } x = v; e \longrightarrow e[x:=v] \\
(\text{match}) & \text{match } (C \ v_1 \dots v_n) \{ \overline{p_i \rightarrow e_i} \} \\
& \longrightarrow e_i[x_1:=v_1, \dots, x_n:=v_n] \\
& \text{with } p_i = C \ x_1 \dots x_n
\end{array}
\end{array}$$

Fig. 6. Standard strict semantics for λ^1 .

match. We assume all patterns in match are mutually exclusive, and all pattern binders are distinct. Syntactic constructs in gray are only generated in derivations of the calculus and are not exposed to users. Among those constructs, dup and drop form the basic instructions of reference counting.

Contexts Δ, Γ are *multisets* containing variable names. We use the compact comma notation for summing (or splitting) multisets. For example, (Γ, x) adds x to Γ , and (Γ_1, Γ_2) appends two multisets Γ_1 and Γ_2 . The set of free variables of an expression e is denoted by $\text{fv}(e)$, and the set of bound variables of a pattern p by $\text{bv}(p)$.

3.2 The Linear Resource Calculus

The derivation $\Delta \mid \Gamma \vdash e \rightsquigarrow e'$ in Figure 5 reads as follows: given a *borrowed environment* Δ , a *linear environment* Γ , an expression e is translated into an expression e' with explicit reference counting instructions. We call variables in the linear environment *owned*.

The key idea of λ^1 is that each resource (i.e., owned variable) is consumed *exactly* once. That is, a resource needs to be explicitly duplicated (in rule DUP) if it is needed more than once; or be explicitly dropped (in rule DROP) if it is not needed. The rules are closely related to linear typing.

Following the key idea, the variable rule VAR consumes a resource when we own and only own x exactly once in the owned environment. For example, to derive the K combinator, $\lambda x \ y. x$, we need to apply DROP to be able to discard y , which gives $\lambda x \ y. \text{drop } y; x$.

The APP rule splits the owned environment Γ into two separate contexts Γ_1 and Γ_2 for expression e_1 and e_2 respectively. Each expression then consumes its corresponding owned environment. Since Γ_2 is consumed in the e_2 derivation, we know that resources in Γ_2 are surely alive when deriving e_1 , and thus we can *borrow* Γ_2 in the e_1 derivation. The rule is quite similar to the [LET!] rule of Wadler’s linear type rules [48, pg.14] where a linear type can be “borrowed” as a regular type during evaluation of a binding.

Borrowing is important as it allows us to conduct a dup as late as possible, or otherwise we will need to duplicate enough resources before we can divide the owned environment. Consider $\lambda f \ g \ x. (f \ x) (g \ x)$. Without borrowing, we have to duplicate x before the application, resulting

in $\lambda f \ g \ x. \text{dup } x; (f \ x) (g \ x)$. With the borrowing environment it is now possible to derive a translation with the dup right before passing x to f : $\lambda f \ g \ x. (f \ (\text{dup } x; x)) (g \ x)$. Notice rule DUP allows dup from the borrowing environment, where DROP only applies to the owned environment.

The LAM rule is interesting as it essentially derives the body of the lambda independently. The premise $\Gamma = \text{fv}(\lambda x. e)$ requires that exactly the free variables in the lambda are owned – this corresponds to the notion that a lambda is allocated as a closure at runtime that holds all free variables of the lambda (and thus the lambda expression *consumes* the free variables). The body of a lambda is evaluated only when applied, so it is derived under an empty borrowed environment only owning the argument and the free variables (in the closure). The translated lambda is also annotated with Γ , as $\lambda^\Gamma x. e$, so we know precisely the resources the lambda should own when evaluated in a heap semantics. We often omit the annotation when it is irrelevant.

The BIND rule is similar to application and borrows Γ_2 in the derivation for the bound expression. This is the main reason to not consider $\text{val } x = e_1; e_2$ as syntactic sugar for $(\lambda x. e_2) e_1$. The MATCH rule consumes the scrutinee and owns the bound variables in each pattern for each branch. For constructors (rule CON), we divide the owned environment into n parts for each component, and allow each component derivation to borrow the owned environment of the components derived later.

We use the notation $\lceil e \rceil$ to erase all drop and dup in the expression e . We can now state that derivations leave expressions unchanged except for inserting dup/drop operations: if $\Delta \mid \Gamma \vdash e \rightsquigarrow e'$ then $e = \lceil e' \rceil$.

3.3 Semantics

Figure 6 defines standard semantics for λ^1 using strict evaluation contexts [49]. The evaluation contexts uniquely determine where to apply an evaluation step. As such, evaluation contexts neatly abstract from the usual implementation context of a stack and program counter. Rule (match) relies on the internal form of expression $\text{match } e \{ \overline{p_i \rightarrow e_i} \}$: after substitution (app), values may appear in positions where only variables were allowed, and this is exactly what enables us to do pattern match on a data constructor.

In Figure 7 we define our target semantics of a reference counted heap, so sharing of values becomes explicit and substitution only substitutes variables. Here, each heap entry $x \mapsto^n v$ points to a value v with a reference count of n (with $n \geq 1$). In these semantics, values other than variables are allocated in the heap with rule (lam_r) and rule (con_r). The evaluation rules discard entries from the heap when the reference count drops to zero. Any allocated lambda is annotated as $\lambda^{ys} x. e$ to clarify that these are essentially *closures* holding an environment ys and a code pointer $\lambda x. e$. Note that it is important that the environment ys is a multi-set. After the initial translation, ys will be equivalent to the free variables

$H :$	$x \rightarrow (\mathbb{N}^+, v)$				
$E ::=$	$\square \mid E \mid E \mid \text{val } x = E; e$				
	$\mid C \ x_1 \dots x_i \ E \ v_j \dots v_n$				
				$\frac{H \mid e \rightarrow_r H' \mid e'}{H \mid E[e] \mapsto_r H' \mid E[e']} \text{ [EVAL]}$	
(lam_r)	$H \mid (\lambda^{ys} x. e)$	$\rightarrow_r$	$H, f \mapsto^1 \lambda^{ys} x. e$	$\mid f$	fresh f
(con_r)	$H \mid C \ x_1 \dots x_n$	$\rightarrow_r$	$H, z \mapsto^1 C \ x_1 \dots x_n$	$\mid z$	fresh z
(app_r)	$H \mid f \ z$	$\rightarrow_r$	$H \mid \text{dup } ys; \text{ drop } f; e[x:=z]$	$(f \mapsto^n \lambda^{ys} x. e) \in H$	
(match_r)	$H \mid \text{match } x \{ \overline{p_i \rightarrow e_i} \}$	$\rightarrow_r$	$H \mid \text{dup } ys; \text{ drop } x; e_i[xs:=ys]$	with $p_i = C \ xs$ and $(x \mapsto^n C \ ys) \in H$	
(bind_r)	$H \mid \text{val } x = y; e$	$\rightarrow_r$	$H \mid e[x:=y]$		
(dup_r)	$H, x \mapsto^n v$	$\mid \text{dup } x; e \rightarrow_r$	$H, x \mapsto^{n+1} v$	$\mid e$	
(drop_r)	$H, x \mapsto^{n+1} v$	$\mid \text{drop } x; e \rightarrow_r$	$H, x \mapsto^n v$	$\mid e$	if $n \geq 1$
(dlam_r)	$H, x \mapsto^1 \lambda^{ys} z. e$	$\mid \text{drop } x; e \rightarrow_r$	$H \mid \text{drop } ys; e$		
(dcon_r)	$H, x \mapsto^1 C \ ys$	$\mid \text{drop } x; e \rightarrow_r$	$H \mid \text{drop } ys; e$		

Fig. 7. Reference-counted heap semantics for λ^1 .

in the body (see rule LAM), but during evaluation substitution may substitute several variables with the same reference. To keep reference counts correct, we need to keep considering each one as a separate entry in the closure environment.

When applying an abstraction, rule (app_r) needs to satisfy the assumptions made when deriving the abstraction in rule LAM . First, the (app_r) rule inserts dup to duplicate variables ys , as these are owned in rule LAM . It then drops the reference to the closure itself. Rule (match_r) is similar to rule (app_r) , which duplicates the newly bound pattern bindings and drops the scrutinee⁵. Rule (bind_r) simply substitutes the bound variable x with the resource y .

Duping a resource is straightforward as rule (dup_r) merely increments the reference count of the resource. Dropping is more involved. Rule (drop_r) just decrements the reference count when there are still multiple copies of it. But when the reference count would drop to zero, rule (dlam_r) and rule (dcon_r) actually *free* a heap entry and then dynamically insert drop operations to drop their fields recursively.

The tricky part of the reference counting semantics is showing *correctness*. We prove this in two parts. First, we prove that the reference counting semantics is *sound* and corresponds to the standard semantics. Below we use heaps as substitutions on expressions. We write $[H]e$ to mean H applied as a substitution to expression e .

Theorem 1. (Reference-counted heap semantics is sound)

If we have $\emptyset \mid \emptyset \vdash e \rightsquigarrow e'$ and $e \mapsto^* v$, then we also have $\emptyset \mid e' \mapsto_r^* H \mid x$ with $[H]x = v$.

⁵A difference between (app_r) and (match_r) is that for application the free variables ys are dynamic and thus the duplication must be done at runtime. In contrast, a match knows the the bound variables in a pattern statically. In practice we therefore generate the required dup and drop operations during elaboration for each branch – this is essential as that enables the further optimizations as shown in Section 2.2.

To prove this theorem we need to maintain strong invariants at each evaluation step to ensure a variable is still alive if it is going to be referred later. Second, we prove that the reference counting semantics never *hold on* to unused variables. We first define the notion of *reachability*.

Definition 1. (Reachability)

We say a variable x is reachable in terms of a heap H and an expression e , denoted as $\text{reach}(x, H \mid e)$, if (1) $x \in \text{fv}(e)$; or (2) for some y , we have $\text{reach}(y, H \mid e) \wedge y \mapsto^n v \in H \wedge \text{reach}(x, H \mid v)$.

With reachability, we can formally show:

Theorem 2. (Reference counting leaves no garbage)

Given $\emptyset; \emptyset \vdash e \rightsquigarrow e'$, and $\emptyset \mid e' \mapsto_r^* H \mid x$, then for every intermediate state $H_i \mid e_i$, we have for all $y \in \text{dom}(H_i)$, $\text{reach}(y, H_i \mid e_i)$.

In the accompanying technical report [41], we further show that the reference counts are exactly equal to the number of actual references to the resource. Notably, to capture the essence of precise reference counting, λ^1 does not model *mutable references* (Section 2.7.3). From Theorem 2 we see that mutable references are indeed the only source of cycles. A natural extension of the system is to include mutable references and thus cycles. In that case, we could generalize Theorem 2, where the conclusion would be that for all resource in the heap, it is either reachable from the expression, or it is part of a cycle.

These theorems establish the correctness of the reference-counted heap semantics. However, correctness does not imply *precision*, ie. that the heap is *garbage free*. Eventually all live data is discarded but it may well hold on to live data too long by delaying drop operations. As an example, consider $y \mapsto^1 () \mid (\lambda x. x) (\text{drop } y; ())$, where y is reachable but dropped too late: it is only dropped after the lambda gets

$\frac{\Delta \mid \Gamma \vdash_s e \rightsquigarrow e'}{\uparrow \quad \uparrow \quad \uparrow \quad \downarrow} \quad \Delta \cap \Gamma = \emptyset \quad \Gamma \subseteq \text{fv}(e) \quad \text{fv}(e) \subseteq \Delta, \Gamma \quad \text{multiplicity of each member in } \Delta, \Gamma \text{ is } 1$	
$\frac{}{\Delta \mid x \vdash_s x \rightsquigarrow x} \text{ [SVAR]}$	$\frac{}{\Delta, x \mid \emptyset \vdash_s x \rightsquigarrow \text{dup } x; x} \text{ [SVAR-DUP]}$
$\frac{\Delta, \Gamma_2 \mid \Gamma - \Gamma_2 \vdash_s e_1 \rightsquigarrow e'_1 \quad \Delta \mid \Gamma_2 \vdash_s e_2 \rightsquigarrow e'_2 \quad \Gamma_2 = \Gamma \cap \text{fv}(e_2)}{\Delta \mid \Gamma \vdash_s e_1 e_2 \rightsquigarrow e'_1 e'_2} \text{ [SAPP]}$	
$\frac{x \in \text{fv}(e) \quad \emptyset \mid ys, x \vdash_s e \rightsquigarrow e' \quad ys = \text{fv}(\lambda x. e) \quad \Delta_1 = ys - \Gamma}{\Delta, \Delta_1 \mid \Gamma \vdash_s \lambda x. e \rightsquigarrow \text{dup } \Delta_1; \lambda^{ys} x. e'} \text{ [SLAM]}$	$\frac{x \notin \text{fv}(e) \quad \emptyset \mid ys \vdash_s e \rightsquigarrow e' \quad ys = \text{fv}(\lambda x. e) \quad \Delta_1 = ys - \Gamma}{\Delta, \Delta_1 \mid \Gamma \vdash_s \lambda x. e \rightsquigarrow \text{dup } \Delta_1; \lambda^{ys} x. (\text{drop } x; e')} \text{ [SLAM-D]}$
$\frac{x \in \text{fv}(e_2) \quad x \notin \Delta, \Gamma \quad \Delta, \Gamma_2 \mid \Gamma - \Gamma_2 \vdash_s e_1 \rightsquigarrow e'_1 \quad \Delta \mid \Gamma_2, x \vdash_s e_2 \rightsquigarrow e'_2 \quad \Gamma_2 = \Gamma \cap (\text{fv}(e_2) - x)}{\Delta \mid \Gamma \vdash_s \text{val } x = e_1; e_2 \rightsquigarrow \text{val } x = e'_1; e'_2} \text{ [SBIND]}$	$\frac{x \notin \text{fv}(e_2), \Delta, \Gamma \quad \Delta, \Gamma_2 \mid \Gamma - \Gamma_2 \vdash_s e_1 \rightsquigarrow e'_1 \quad \Delta \mid \Gamma_2 \vdash_s e_2 \rightsquigarrow e'_2 \quad \Gamma_2 = \Gamma \cap \text{fv}(e_2)}{\Delta \mid \Gamma \vdash_s \text{val } x = e_1; e_2 \rightsquigarrow \text{val } x = e'_1; \text{drop } x; e'_2} \text{ [SBIND-D]}$
$\frac{\Delta \mid \Gamma_i \vdash_s e_i \rightsquigarrow e'_i \quad \Gamma_i = (\Gamma, \text{bv}(p_i)) \cap \text{fv}(e_i) \quad \Gamma'_i = (\Gamma, \text{bv}(p_i)) - \Gamma_i}{\Delta \mid \Gamma, x \vdash_s \text{match } x \{ p_i \mapsto e_i \} \rightsquigarrow \text{match } x \{ p_i \mapsto \text{drop } \Gamma'_i; e'_i \}} \text{ [SMATCH]}$	
$\frac{\Delta, \Gamma_{i+1}, \dots, \Gamma_n \mid \Gamma_i \vdash_s v_i \rightsquigarrow v'_i \quad 1 \leq i \leq n \quad \Gamma_i = (\Gamma - \Gamma_{i+1} - \dots - \Gamma_n) \cap \text{fv}(v_i)}{\Delta \mid \Gamma \vdash_s C v_1 \dots v_n \rightsquigarrow C v'_1 \dots v'_n} \text{ [SCON]}$	

Fig. 8. Syntax-directed linear resource rules of λ^1 .

allocated. In contrast, a *garbage free* algorithm would produce $y \mapsto^1 () \mid \text{drop } y; (\lambda x. x) ()$. In the next section we present Perceus as a syntax directed algorithm of the linear resource calculus and show that it is *garbage free*.

3.4 Perceus

Figure 8 defines syntax directed derivation $\vdash_s$ for our resource calculus and as such specifies our *Perceus algorithm*. Like before, $\Delta \mid \Gamma \vdash_s e \rightsquigarrow e'$ translates an expression e to e' under an borrowed environment Δ and an owned environment Γ . During the derivation, we maintain the following invariants: (1) $\Delta \cap \Gamma = \emptyset$; (2) $\Gamma \subseteq \text{fv}(e)$; (3) $\text{fv}(e) \subseteq \Delta, \Gamma$; and (4) multiplicity of each member in Δ, Γ is 1. We ensure these properties hold by construction at any step in a derivation.

The Perceus rules are set up to do precise reference counting: we delay a *dup* operation to come as late as possible, pushing them out to the leaves of a derivation; and we generate a *drop* operation as soon as possible, right after a binding or at the start of a branch.

Rule SVAR-DUP borrows x by inserting a *dup*. The SAPP rule now deterministically finds a good split of the environment Γ . We pass the intersection of Γ with the free variables in e_2 to the e_2 derivation. Otherwise the rule is the same as in the declarative system. For abstraction and binding we have two variants: one where the binding is actually in the free variables of the expression (rule SLAM and SBIND), and one where the binding can be immediately dropped as it is unused (rule SLAM-D and SBIND-D). In the abstraction rule, we

know that $\Gamma \subseteq \text{fv}(\lambda x. e)$ and thus $\Gamma \subseteq ys$. If there are any free variables not in Γ , they must be part of the borrowed environment (as Δ_1) and these must be duplicated to ensure ownership. The bind rules are similarly constructed as a mixture of SAPP and SLAM.

The SMATCH rule is interesting as in each branch there may be variables that can to be dropped as they no longer occur as free variables in that branch. The owned environment Γ_i in the i th branch is the intersection of $(\Gamma, \text{bv}(p_i))$ and the free variables in that branch; any other owned variables (as Γ'_i) are dropped at the start of the branch. Rule SCON deterministically splits the environment Γ as in rule SAPP.

We show that the Perceus algorithm is sound by showing that for each rule there exists a derivation in the declarative linear resource calculus.

Theorem 3. (Syntax directed translation is sound.)

If $\Delta \mid \Gamma \vdash_s e \rightsquigarrow e'$ then also $\Delta \mid \Gamma \vdash e \rightsquigarrow e'$.

More importantly, we prove that any translation resulting from the Perceus algorithm is *precise*, where any intermediate state in the evaluation is *garbage free*:

Theorem 4. (Perceus is precise and garbage free)

If $\emptyset \mid \emptyset \vdash_s e \rightsquigarrow e'$ and $\emptyset \mid e' \mapsto_r^* H \mid x$, then for every intermediate state $H_i \mid e_i$ that is not at a *dup*/*drop* operation ($e_i \neq E[\text{drop } x; e'_i]$ and $e_i \neq E[\text{dup } x; e'_i]$), we have that for all $y \in \text{dom}(H_i)$, $\text{reach}(y, H_i \mid \lceil e_i \rceil)$.

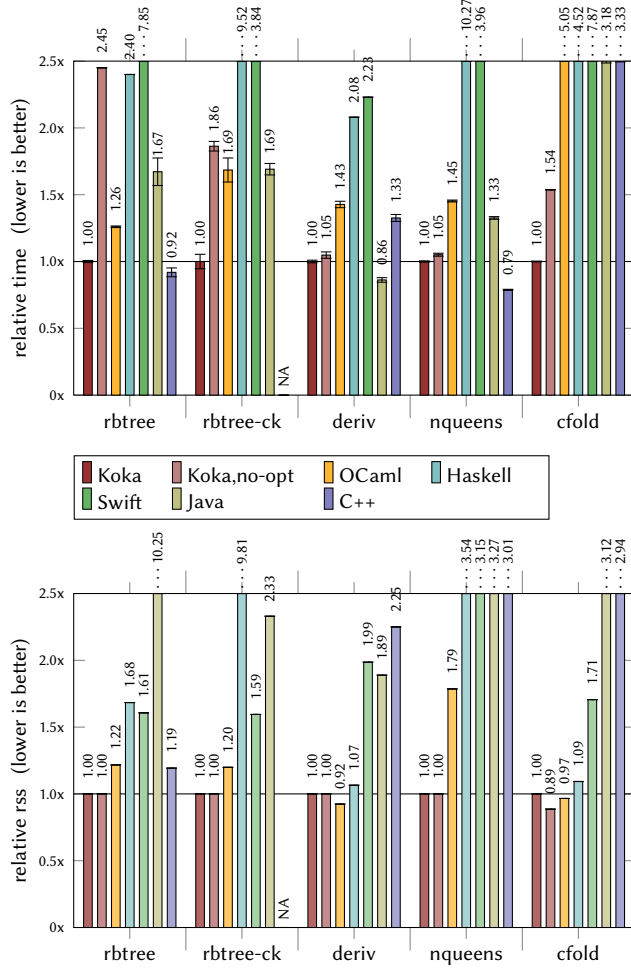


Fig. 9. Relative execution time and peak working set with respect to Koka. Using a 6-core 64-bit AMD 3600XT 3.8Ghz with 64GiB 3600Mhz memory, Ubuntu 20.04.

This theorem states that after evaluating any immediate reference counting instructions, every variable in the heap is reachable from the *erased* expression. This rules out, for example, $y \mapsto^1 () \mid (\lambda x. x) (\text{drop } y; ())$ as y is not in the free variables of the erased expression. Just like Theorem 2, if the system is extended with mutable references, then Theorem 4 could be generalized such that every resource is either reachable from the erased expression, or it is part of a cycle.

The implementation of Perceus is further extended with the optimizations described in Section 2. As the component transformations, including inlining and dup/drop fusion, are standard, the soundness of those optimizations follows naturally and a proof is beyond the scope of this paper.

4 Benchmarks

In this section we discuss initial benchmarks of Perceus as implemented in Koka, versus state-of-the-art memory reclamation implementations in various other languages. Since

we compare across languages we need to interpret the results with care – the results depend not only on memory reclamation but also on the different optimizations performed by each compiler and how well we can translate each benchmark to that particular language. We view these results therefore mostly as *evidence that the Perceus reference counting technique is viable and can be competitive* and not as a direct comparison of absolute performance between systems.

As such, we selected only benchmarks that stress memory allocation, and we tried to select mature comparison systems that use a range of memory reclamation techniques and are considered best-in-class. The systems we compare are:

- Koka 2.0.3, compiling the generated C code with gcc 9.3.0 using a customized version of the mimalloc allocator [27]. We also run Koka “no-opt” with drop/reuse specialization and reuse analysis disabled to measure the impact of those optimizations.
- OCaml 4.08.1. This has a stop-the-world generational collector with a minor and major heap. The minor heap uses a copying collector, while a tracing collector is used for the major heap [11, 34, Chap.22]. The Koka benchmarks correspond essentially one-to-one to the OCaml versions.
- Haskell, GHC 8.6.5. A highly optimizing compiler with a multi generational garbage collector. The benchmark sources again correspond very closely, but since Haskell has lazy semantics, we used strictness annotations in the data structures to speed up the benchmarks, as well as to ensure that the same amount of work is done.
- Swift 5.3. The only other language in this comparison where the compiler uses reference counting [6, 47]. The benchmarks are directly translated to Swift in a functional style without using direct mutation. However, we translated tail-recursive definitions to explicit loops with local variables.
- Java SE 15.0.1. Uses the HotSpot JVM and the G1 concurrent, low-latency, generational garbage collector. The benchmarks are directly translated from Swift.
- C++, gcc 9.3.0 using the standard libc allocator. A highly optimizing compiler with manual memory management. Without automatic memory management, many benchmarks are difficult to express directly in C++ as they use persistent and partially shared data structures. To implement these faithfully would essentially require manual reference counting. Instead, we use C++ as our performance baseline: if provided, we either use in-place updates without supporting persistence (as in `rbtree` which uses `std::map`) or we do not reclaim memory at all (as in `deriv`, `nqueens`, and `cfold`).

The benchmarks are all chosen to be medium sized and non-trivial, and all stress memory allocation with little computation. Most of these are based on the benchmark suite of Lean [46] and all are available in the Koka repository [1]. The execution times and peak working set as the median over 10 runs and normalized to Koka are given in Figure 9

(each benchmark runs between 1 to 5 seconds for Koka, and uses up to 300MiB). When a benchmark is not available for a particular language, it is marked as “NA” in the figures.

- **rbtree**: this benchmark performs 42 million insertions into a red-black balanced tree and after that folds over the tree counting the true elements. Here the reuse analysis of Koka (as shown in Section 2.4) is doing well compared to the other systems. OCaml is close in performance – rebalancing generates lots of short-lived object allocation which are a great fit a minor heap copying-collector with fast aggregated bump-pointer allocation. The C++ benchmark is implemented using the in-place updating `std::map` implementation, which internally uses an optimized red-black tree implementation [13]. Surprisingly, the purely functional Koka implementation is within 10% of the C++ performance. Since the insertion operations are the same, we believe this is partly because C++ allocations must be 16-byte aligned while the Koka allocator can use 8-byte alignment in the allocations and thus allocate a bit less (as apparent in Figure 9) (and similarly, bump pointer allocation in OCaml can be faster than general `malloc/free`). Java performs close to C++ here but also uses almost 10× the memory of Koka (1.7GiB vs. 170MiB, Figure 9). This can be reduced to about 1.5× by providing tuning parameters on the command line but that also made it slower on our system. This benchmark also shows the potential effectiveness of the reference count optimizations where the “no-opt” version is more than 2× slower. However, in benchmarks with lots of sharing, like `deriv` and `nqueens`, the optimizations are less effective. More generally, we expect a GC to do better when reuse optimization is not triggered, and there is lots of short-lived object allocation.
- **rbtree-ck**: it has been suggested that `rbtree` is biased to reference counting as it has no shared subtrees and thus reuse analysis can use in-place updates all the time. The `rbtree-ck` benchmark remedies this and is a variant of `rbtree` that keeps a list of every 5th tree generated and thus shares many subtrees. This pattern occurs often in practice, for example in compilers using scoped environments, or in backtracking searches where the original state is shared among different exploratory branches. Again though the reference counting strategy outperforms all other systems. Haskell and OCaml are now relatively slower than in `rbtree` – we conjecture this is due to extra copying between generations, and perhaps due to increased tracing cost. We have no C++ version of this benchmark as that would essentially require a persistent implementation of `std::map`.
- **deriv**: calculates the derivative of large symbolic expressions (up to 10M nodes). Interestingly, the memory usage of OCaml is slightly less here than Koka – since Perceus is *garbage free* we would expect though that Koka *always*

uses less memory than a GC based system. From studying the generated code of OCaml we believe that it is because the optimizing OCaml compiler can avoid some allocations by applying inlining with “case of case” transformations [38] which the naive Koka compiler is not (yet) doing. It is also interesting to see that the “no-opt” Koka is only just slightly slower than optimized Koka here. This is probably due to the sharing of many sub-expressions when calculating the derivative – this in turn causes the code resulting from drop/reuse specialization and reuse analysis to mostly use the “slow” path which is equivalent to the one in “no-opt”.

- **nqueens**: calculates all solutions for the n-queens problem of size 13 into a list, and returns the length of that list. The solution lists share many sub-solutions and, as in `deriv`, for the C++ version we do *not* free any memory (but do allocate the same objects as the other benchmarks). Again, Koka is quite competitive even with the large amount of shared structures, and the peak working set is significantly lower.
- **cfold**: performs constant-folding over a large symbolic expression (2M nodes). This benchmark is similar to the `deriv` benchmark and manipulates a complex expression graph. Koka does significantly better than other systems. Just as in `deriv`, we see that OCaml uses slightly less memory as it can avoid some allocations by optimizing well. The “no-opt” version of Koka also uses 11% less memory; this is because the reuse analysis essentially holds on to memory for later reuse. Just like with scoped based reference counting that may lead to increased memory usage in some situations.

An interesting overall observation is that the reference counting implementation of Swift seems less effective than Koka – this may be partly due to the language and compiler, but we also believe that this may be a confirmation of our initial hypothesis where we argue that a combination of static compiler optimizations with dynamic runtime checks (e.g. `is-unique`) are needed for best results. As discussed for example in Section 2.7.2, some of the optimizations we perform are difficult to do in Swift as the static guarantees of the language are not strong enough. More research is needed though to confirm this as there may be other causes well unrelated to reference counting as such.

Finally, we also ran our benchmarks using just atomic operations for our reference counts to see the impact of the thread-shared flag. We observed a slowdown from 5% (`rbtree`) up to 59% (`nqueens`) across our benchmarks. This matches the observations by Ungar et al. [47] who performed a similar experiment in Swift.

5 Related Work

Our work is closely based on the reference counting algorithm in the Lean theorem prover as described by Ullrich and de Moura [46]. They describe reuse analysis based on

reset/reuse instructions, and describe both reference counting based on ownership (i.e. precise) but also support borrowed parameters. We extend their work with drop- and reuse specialization, and generalize to a general purpose language with side-effects and complex control flow. We also introduce a novel formalization of reference counting with the linear resource calculus, and define our algorithm in terms of that. As such, the Perceus algorithm may differ from the Lean one as that is specified over a lower-level calculus that uses explicit partial application nodes (pap) and has no first-class lambda expressions. Schulte [43] describes an algorithm for inserting reference count instructions in a small first-order language and shows a limited form of reuse analysis, called “reusage” (transformation T14).

Using explicit reference count instructions in order to optimize them via static analysis is described as early as Barth [3]. Mutating unique references in place has traditionally focused on array updates [19], as in functional array languages like Sisal [32] and SaC [16, 42]. Férey and Shankar [12] provide functional array primitives that use in-place mutation if the array has a unique reference; we plan to add these to Koka. We believe this would work especially well in combination with reuse-analysis for BTree-like structures using trees of small functional arrays.

The λ^1 calculus is closely based on linear logic. Turner and Wadler [45] give a heap-based operational interpretation which does not need reference counts as linearity is tracked by the type system. In contrast, Chirimar et al. [5] give an interpretation of linear logic in terms of reference counting, but in their system, values with a linear type are not guaranteed to have a unique reference at runtime.

Generally, a system with linear types [48], like linear Haskell [4], or the uniqueness typing of Clean [2, 8], can offer *static* guarantees that the corresponding objects are unique at runtime, so that destructive updates can always be performed safely. However, this usually also requires writing multiple versions of a function for each case (unique-versus shared argument). By contrast, reuse analysis relies on dynamic runtime information, and thus reuse can be performed generally. This is also what enables FBIP to use a single function that can be used for both unique or shared objects (since the uniqueness property is *not* part of the type). These two mechanisms could be combined: if our system is extended with unique types, then reuse analysis could statically eliminate corresponding uniqueness checks.

The Swift language is widely used in iOS development and uses reference counting with an explicit representation in its intermediate language. There is no reuse analysis but, as remarked by Ullrich and de Moura [46], this may not be so important for Swift as typical programs mutate objects in-place. There is no cycle collection for Swift, but despite the widespread usage of mutation this seems to be not a

large problem in practice. Since it can be easy to create accidental cycles through the self pointer in callbacks, Swift has good support for *weak* references to break such cycles in a declarative manner. Ungar et al. [47] optimize atomic reference counts by tagging objects that can be potentially thread-shared. Later work by Choi et al. [6], uses *biased* reference counting to avoid many atomic updates.

The CPython implementation also uses reference counting, and uses ownership-based reference counts for parameters but still only drops the reference count of local variables when exiting the frame. Another recent language that uses reference counting is Nim. The reference counting method is scope-based and uses non-atomic operations (and objects cannot be shared across threads without extra precautions). Nim can be configured to use ORC reference counting which extends the basic ARC collector with a cycle collection [53]. Nim has the *acyclic* annotation to identify data types that are (co)-inductive, as well as the (unsafe) *cursor* annotation for variables that should not be reference counted.

In our work we focus on *precise* and *garbage free* reference counting which enables static optimization of reference count instructions. On the other extreme, Deutsch and Bobrow [10] consider *deferred* reference counting – any reference count operations on stack-based local variables are *deferred* and only the reference counts of fields in the heap are maintained. Much like a tracing collector, the stack roots are periodically scanned and deferred reference counting operations are performed. Levanoni and Petrank [28] extend this work and present a high performance reference counting collector for Java that uses the *sliding view* algorithm to avoid many intermediate reference counting operations and needs no synchronization on the write barrier.

6 Conclusion and Future Work

In this paper we present Perceus, a precise reference counting system with reuse and specialization, which is built upon λ^1 , a novel linear resource calculus closely based on linear logic. Our implementation in Koka is competitive with other mature memory collectors over our benchmark suite but more experimentation in larger systems is needed. We would like to integrate selective “borrowing” into Perceus – this would make certain programs no longer be *garbage free*, but we believe it could deliver further performance improvements if judiciously applied. It also remains to be seen how to handle cycle collection efficiently. Finally, the explicit control-flow is not zero-cost (like C++ exception handling), and it would be interesting to see if this can be improved further.

Acknowledgements

We like to thank Erez Petrank for the discussions on the race conditions that can occur with in-place updates and reference counts. Alex Reinking was supported by the United States National Science Foundation under Grant 1723445.

References

- [1] Koka repository. 2019. URL <https://github.com/koka-lang/koka>.
- [2] Erik Barendsen and Sjaak Smetsers. Uniqueness typing for functional languages with graph rewriting semantics. *Mathematical Structures in Computer Science*, 6 (6): 579–612, 1996. <https://doi.org/10.1017/S0960129500070109>.
- [3] Jeffrey M. Barth. Shifting garbage collection overhead to compile time. Technical Report UCB/ERL M524, EECS Department, University of California, Berkeley, Jun 1975. URL <http://www2.eecs.berkeley.edu/Pubs/TechRpts/1975/29109.html>.
- [4] Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. Linear haskell: Practical linearity in a higher-order polymorphic language. *Proc. ACM Program. Lang.*, 2 (POPL), December 2017. <https://doi.org/10.1145/3158093>.
- [5] Jawahar Chirimar, Carl A. Gunter, and Jon G. Riecke. Reference counting as a computational interpretation of linear logic. *Journal of Functional Programming*, 6: 6–2, 1996.
- [6] Jiho Choi, Thomas Shull, and Josep Torrellas. Biased reference counting: Minimizing atomic operations in garbage collection. In *Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques*, PACT '18, 2018. <https://doi.org/10.1145/3243176.3243195>.
- [7] George E Collins. A method for overlapping and erasure of lists. *Communications of the ACM*, 3 (12): 655–657, 1960.
- [8] Edsko de Vries, Rinus Plasmeijer, and David M. Abrahamson. Uniqueness typing simplified. In Olaf Chitil, Zoltán Horváth, and Viktória Zsóka, editors, *Implementation and Application of Functional Languages (IFL'08)*, pages 201–218. Springer, 2008. ISBN 978-3-540-85373-2.
- [9] David L. Detlefs, Paul A. Martin, Mark Moir, and Guy L. Steele Jr. Lock-free reference counting. In *Proceedings of the 20th Annual ACM Symposium on Principles of Distributed Computing*, pages 190–199, 2001.
- [10] L. Peter Deutsch and Daniel G. Bobrow. An efficient, incremental, automatic garbage collector. *Communications of the ACM*, 19 (9): 522–526, September 1976. ISSN 0001-0782. <https://doi.org/10.1145/360336.360345>.
- [11] Damien Doligez and Xavier Leroy. A concurrent, generational garbage collector for a multithreaded implementation of ML. In *Proceedings of the 20th ACM Symposium on Principles of Programming Languages (POPL)*, pages 113–123. ACM press, January 1993.
- [12] Gaspard Férey and Natarajan Shankar. Code generation using a formal model of reference counting. In Sanjai Rayadurgam and Oksana Tkachuk, editors, *NASA Formal Methods*, pages 150–165. Springer International Publishing, 2016. ISBN 978-3-319-40648-0.
- [13] Free Software Foundation, Silicon Graphics, and Hewlett-Packard Company. Internal red-black tree implementation for “stl::map”. URL <https://code.woboq.org/gcc/libstdc++-v3/src/c++98/tree.cc.html>.
- [14] Matt Gallagher. Reference counted releases in Swift. Blog post, December 2016. URL <https://www.cocoawithlove.com/blog/resources-releases-reentrancy.html>.
- [15] A. Gidenstam, M. Papatrifiou, H. Sundell, and P. Tsigas. Efficient and reliable lock-free memory reclamation based on reference counting. In *8th International Symposium on Parallel Architectures, Algorithms and Networks (ISPAN'05)*, 2005.
- [16] Clemens Grelck and Kai Trojahn. Implicit memory management for SAC. In *6th International Workshop on Implementation and Application of Functional Languages (IFL'04)*, September 2004.
- [17] Leo J Guibas and Robert Sedgewick. A dichromatic framework for balanced trees. In *19th Annual Symposium on Foundations of Computer Science (sfcs 1978)*, pages 8–21. IEEE, 1978.
- [18] Carl A. Gunter, Didier Rémy, and Jon G. Riecke. A generalization of exceptions and control in ml-like languages. In *Proceedings of the Seventh International Conference on Functional Programming Languages and Computer Architecture*, FPCA '95, page 12–23. ACM, 1995. ISBN 0897917197. <https://doi.org/10.1145/224164.224173>.
- [19] Paul Hudak and Adrienne Bloss. The aggregate update problem in functional programming systems. In *Proceedings of the 12th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, POPL '85, page 300–314. ACM, 1985. ISBN 0897911474. <https://doi.org/10.1145/318593.318660>.
- [20] Gérard P. Huet. The zipper. *Journal of Functional Programming*, 7 (5): 549–554, 1997.
- [21] Apple Inc. The Swift guide: Error handling. 2017. URL <https://docs.swift.org/swift-book/LanguageGuide/ErrorHandling.html>.
- [22] Donald E. Knuth. *The Art of Computer Programming, Volume 1 (3rd Ed.): Fundamental Algorithms*. Addison Wesley Longman Publishing Co., Inc., 1997. ISBN 0201896834.
- [23] Daan Leijen. Koka: Programming with row polymorphic effect types. In *MSFP'14, 5th workshop on Mathematically Structured Functional Programming*, 2014. <https://doi.org/10.4204/EPTCS.153.8>.
- [24] Daan Leijen. Algebraic effects for functional programming. Technical Report MSR-TR-2016-29, Microsoft Research technical report, August 2016. Extended version of [25].
- [25] Daan Leijen. Type directed compilation of row-typed algebraic effects. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL'17)*, pages 486–499, January 2017a. ISBN 978-1-4503-4660-3. <https://doi.org/10.1145/3009837.3009872>.
- [26] Daan Leijen. Structured asynchrony with algebraic effects. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on Type-Driven Development*, TyDe 2017, pages 16–29, 2017b. ISBN 978-1-4503-5183-6. <https://doi.org/10.1145/3122975.3122977>.
- [27] Daan Leijen, Zorn Ben, and Leo de Moura. Mimalloc: Free list sharding in action. *Programming Languages and Systems*, 11893, 2019. https://doi.org/10.1007/978-3-030-34175-6_13. APLAS'19.
- [28] Yossi Levani and Erez Petrank. An on-the-fly reference-counting garbage collector for java. *ACM Trans. Program. Lang. Syst.*, 28 (1): 1–69, January 2006. ISSN 0164-0925. <https://doi.org/10.1145/1111596.1111597>.
- [29] Prabhakar Mateti and Ravi Manghmalani. Morris' tree traversal algorithm reconsidered. *Science of Computer Programming*, 11 (1): 29–43, 1988. ISSN 0167-6423. [https://doi.org/10.1016/0167-6423\(88\)90063-9](https://doi.org/10.1016/0167-6423(88)90063-9).
- [30] Conor McBride. The derivative of a regular type is its type of one-hole contexts, 2001. URL <http://strictlypositive.org/diff.pdf>. (Extended Abstract).
- [31] John McCarthy. Recursive functions of symbolic expressions and their computation by machine, part i. *Communications of the ACM*, 3 (4): 184–195, 1960.
- [32] J. McGraw, S. Skedzielewski, S. Allan, D. Grit, R. Oldehoeft, J. Glauert, I. Dobes, and P. Hohensee. SISAL: streams and iteration in a single-assignment language. language reference manual, version 1.1. Technical Report LLL/M-146, ON: DE83016576, Lawrence Livermore National Lab., CA, USA, 7 1983.
- [33] Maged M. Michael. Hazard pointers: Safe memory reclamation for lock-free objects. *IEEE Trans. Parallel Distrib. Syst.*, 15 (6): 491–504, June 2004. <https://doi.org/10.1109/TPDS.2004.8>.
- [34] Yaron Minsky, Anil Madhavapeddy, and Jason Hickey. *Real World OCaml: Functional programming for the masses*. 2012. ISBN 978-1449323912. URL <https://dev.realworldocaml.org>.
- [35] Joseph M. Morris. Traversing binary trees simply and cheaply. *Information Processing Letters*, 9 (5): 197 – 200, 1979. [https://doi.org/10.1016/0020-0190\(79\)90068-1](https://doi.org/10.1016/0020-0190(79)90068-1).
- [36] Chris Okasaki. *Purely Functional Data Structures*. Columbia University, June 1999a. ISBN 9780521663502.
- [37] Chris Okasaki. Red-black trees in a functional setting. *Journal of Functional Programming*, 9 (4): 471–477, 1999b. <https://doi.org/10.1017/S0956796899003494>.
- [38] Simon L. Peyton Jones and André L. M. Santos. A transformation-based optimiser for haskell. *Science of Computer Programming*, 32 (1):

- 3 – 47, 1998. [https://doi.org/10.1016/S0167-6423\(97\)00029-4](https://doi.org/10.1016/S0167-6423(97)00029-4).
- [39] Gordon D. Plotkin and John Power. Algebraic operations and generic effects. *Applied Categorical Structures*, 11 (1): 69–94, 2003. <https://doi.org/10.1023/A:1023064908962>.
- [40] Gordon D. Plotkin and Matija Pretnar. Handling algebraic effects. volume 9, 2013. [https://doi.org/10.2168/LMCS-9\(4:23\)2013](https://doi.org/10.2168/LMCS-9(4:23)2013).
- [41] Alex Reinking, Ningning Xie, Leonardo de Moura, and Daan Leijen. Perceus: Garbage free reference counting with reuse. Technical Report MSR-TR-2020-42, Microsoft Research, November 2020.
- [42] Sven-Bodo Scholz. Single Assignment C: Efficient support for high-level array operations in a functional setting. *Journal of Functional Programming*, 13 (6): 1005–1059, November 2003. <https://doi.org/10.1017/S0956796802004458>.
- [43] Wolfram Schulte. Deriving residual reference count garbage collectors. In Manuel Hermenegildo and Jaan Penjam, editors, *Programming Language Implementation and Logic Programming (PLILP)*, pages 102–116, Berlin, Heidelberg, 1994. Springer Berlin Heidelberg. ISBN 978-3-540-48695-4.
- [44] Herb S. Sutter. Zero-overhead deterministic exceptions: Throwing values. C++ open-std proposal P0709 R2, 10 2018.
- [45] David N. Turner and Phillip Wadler. Operational interpretations of linear logic. (227): 231–248, 1999.
- [46] Sebastian Ullrich and Leonardo de Moura. Counting immutable beans – reference counting optimized for purely functional programming. In *Proceedings of the 31st symposium on Implementation and Application of Functional Languages (IFL '19)*, September 2019.
- [47] David Ungar, David Grove, and Hubertus Franke. Dynamic atomicity: Optimizing swift memory management. In *Proceedings of the 13th ACM SIGPLAN International Symposium on on Dynamic Languages*, DLS 2017, page 15–26, 2017. <https://doi.org/10.1145/3133841.3133843>.
- [48] Phillip Wadler. Linear types can change the world! In *Programming Concepts and Methods*, 1990.
- [49] Andrew K. Wright and Matthias Felleisen. A syntactic approach to type soundness. *Inf. Comput.*, 115 (1): 38–94, November 1994. <https://doi.org/10.1006/inco.1994.1093>.
- [50] Ningning Xie and Daan Leijen. Effect handlers in Haskell, evidently. In *Proceedings of the 2020 ACM SIGPLAN Symposium on Haskell*, Haskell'20, August 2020. <https://doi.org/10.1145/3406088.3409022>.
- [51] Ningning Xie and Daan Leijen. Generalized evidence passing for effect handlers. Technical Report MSR-TR-2021-5, Microsoft Research, March 2021.
- [52] Ningning Xie, Jonathan Brachthäuser, Phillip Schuster, Daniel Hillerström, and Daan Leijen. Effect handlers, evidently. In *Proceedings of the 25th ACM SIGPLAN International Conference on Functional Programming (ICFP'2020)*, ICFP '20, August 2020. <https://doi.org/10.1145/3408981>.
- [53] Danil Yarantsev. Orc - nim's cycle collector. October 2020. URL <https://nim-lang.org/blog/2020/10/15/introduction-to-arc-orc-in-nim.html>.